

UAI301 Object-Oriented Programming
Lab Assignment 4
Topic: Classes, Objects, Constructors, and Methods

Q1. Printing a Class Object

Consider the following class:

```
class MyClass:  
    x = 5
```

Write a Python program to:

1. Print MyClass.
2. Access and print the value of x using MyClass.x.

Q2. Class Variables and Instance Variables

Consider the scenario of **1000 students in TIET**. Identify the information that is common to all students and the information that is different for every student.

Write a Python program using a Student class with:

- Class variables: college and country
- Instance variables: name and roll

Create two student objects and display their details.

Q3. Proving that a Class is an Object

Write a Python program that demonstrates that a class itself is an object.

Use:

```
class Student:  
    college = "TIET"
```

Then:

1. Print Student.
2. Print type(Student).
3. Print Student.college **without creating any student object**.

Q4. Accessing a Class Variable in Different Ways

Consider the Instagram class with the class variable:

```
app_version = "2.5.1"
```

Write a Python program to access app_version using:

1. The class name.
2. A temporary object.
3. A named object.

Display the output for all three cases.

Q5. Class Variable – Netflix Example

Create a Netflix class having a class variable:

```
subscription = "Premium"
```

Demonstrate accessing this variable using:

1. The class name.
2. A temporary object.
3. A named object.

Display the output for all three cases.

Q6. Accessing and Modifying Class and Instance Variables

Develop an **Employee Management System** using a Python class Employee.

The class should contain:

- Class variable: company = "Google"
- Instance variables: name and salary

Create two employee objects, emp1 and emp2.

Perform the following operations:

1. Display the initial company, employee names, and salaries.
2. Modify the salary of emp1.
3. Modify the class variable company.
4. Display the values after modification.
5. Observe which changes affect one object and which changes affect all objects.

Q7. Class, Object and type()

Write a Python program using the Student class to demonstrate:

1. Printing the class using print(Student).
2. Finding the type of the class using type(Student).
3. Creating two objects s1 and s2.
4. Printing the type of both objects.
5. Accessing the class variable college.
6. Accessing the instance variables name and roll.

Observe the difference between the output of print(Student) and print(s1).

Q8. Default Arguments and None in a Constructor

Consider the constructor:

```
def __init__(self, state, voters=None, leader=None):
```

Write a Python program using the Election class to create the following objects:

```
punjab = Election("Punjab", 20_000_000)
up = Election("Uttar Pradesh", 150_000_000)
bjp = Election("Bihar", 20000, "Aman")
```

Perform the following:

1. Display the voters of Punjab.
2. Display the voters of Uttar Pradesh.
3. Display the leader of Bihar.
4. Modify the leader of the Bihar object.
5. Observe why no error occurs when leader is not supplied for Punjab and Uttar Pradesh.
6. Observe the role of None as a default argument.

Q9. Mandatory vs Optional Constructor Arguments

Modify the Election constructor conceptually as follows:

```
def __init__(self, state, voters, leader):
```

Attempt to create an object using:

```
punjab = Election("Punjab", 200000000)
```

Observe the error and see why it occurs.

Compare this with:

```
def __init__(self, state, voters=None, leader=None):
```

and observe the difference between **mandatory and optional constructor parameters**.

Q10. Create a class Patient with attributes: Name, Age, and Blood Group. Initialize the attributes using a constructor and display the details using a member function.

Q11. Create a class Student. Input: Name and Marks in four subjects. Create a method to calculate and display the Average Marks and Grade (e.g., Grade A if average ≥ 75 , else Grade B).

Q12. Create a class BankAccount.

Store: Account Holder Name, Account Number, and Balance.

Create methods: Deposit Amount (increases balance), Withdraw Amount (decreases balance if sufficient funds are available), and Display Balance.

Q13. Create an InventoryItem class with attributes item_name, sku, price, and stock_quantity. Implement methods to display item details, purchase item (reduce stock), and restock item (increase stock).

Q14. Create a FitnessTracker class with attributes user_name, daily_step_goal, and steps_walked. Implement methods to calculate the progress percentage towards the daily step goal and display whether the user has achieved their fitness goal for the day.

Q15. Case Study 1: Smart Home Energy Management System

You are designing software for a Smart Home Hub. Every connected appliance needs to be represented as an object to track its operating status and power usage.

- a. **Define a Class:** Create a class named SmartAppliance.
- b. **Constructor Initialization:** Use the `__init__` constructor method to initialize the following instance attributes:
 - appliance_name (string, e.g., "Air Conditioner")
 - room_location (string, e.g., "Living Room")
 - power_rating_watts (float or int, e.g., 1500)
 - is_on (boolean, defaults to False upon creation)
- c. **Member Methods:**
 - turn_on(): Sets is_on to True and displays a confirmation message.
 - turn_off(): Sets is_on to False and displays a confirmation message.
 - display_status(): Prints all details of the appliance, including its current state (ON/OFF).
- d. **Execution:**
 - Instantiate two separate SmartAppliance objects (e.g., ac_unit and smart_light).
 - Turn one unit ON, display status for both objects, and confirm that their state values are managed independently.

Q16. A smart city uses distribution substations to supply electricity to local neighborhoods. To track energy usage across the entire city, the Smart Energy Board wants a simple Python program to monitor individual substations and track total citywide power consumption.

Requirements

1. **Class Definition & Class Variables:**
 - Create a class named EnergySubstation.
 - Define a **class variable** named city_name set to "Neo Metro".
 - Define a **class variable** named total_city_power_drawn initialized to 0.0 (tracks power drawn across all substations).
2. **Constructor (`__init__` method):**
 - Initialize each object with the following **instance variables**:
 - substation_id (string)
 - max_capacity_kw (float)
 - current_load_kw (float, initialized to 0.0)
3. **Instance Methods (using self):**
 - **add_load(amount):**
 - Checks if adding amount exceeds self.max_capacity_kw.
 - If it does **not** exceed capacity:
 - Increases self.current_load_kw by amount.

- Increases the class variable `EnergySubstation.total_city_power_drawn` by amount.
 - Prints: "[substation_id]: Added [amount] kW load. Current load: [current_load_kw]/[max_capacity_kw] kW."
 - If it exceeds capacity:
 - Prints: "[substation_id]: Overload warning! Cannot add [amount] kW load."
 - **display_status():**
 - Prints the station's ID, current load, max capacity, and city name.
- 4. **Accessing & Modifying Properties:**
 - Create two substation objects:
 - sub_1 with ID "SUB_NORTH" and capacity 500.0 kW.
 - sub_2 with ID "SUB_SOUTH" and capacity 300.0 kW.
 - Add a load of 350.0 kW to sub_1.
 - Add a load of 200.0 kW to sub_2.
 - Attempt to add another 150.0 kW to sub_2 (this should trigger an overload warning).
 - Manually modify the `max_capacity_kw` of sub_2 directly to 400.0 kW to expand its capacity.
 - Add 100.0 kW to sub_2 again.
 - Print `EnergySubstation.total_city_power_drawn` to verify the total citywide grid consumption.